

Policy Management System

Contents

1.0	Problem statement	2
2.0	Architecture Diagram	3
3.0	Tool Chain	5
4.0	User Stories	6
5.0	Proposed Rest Endpoints to be exposed	10
5.1	Rest APIs:	10
6.0	Rubrics/Expected Deliverables	11
6.1	Engineering Concepts (Compute & Integration):	11
6.2	Engineering Concepts (Security & Identity):	11
6.3	Products & Framework (Database & Storage):	11
6.4	Products & Framework (Governance & tooling):	11
6.5	Debugging & Troubleshooting	12
6.6	Code Quality/Optimizations	12
7.0	FrontEnd	12
8.0	Platform	12
8.1	Compute	12
8.2	Compute, Identity & Compliance, Security& Content Delivery	12
9.0	Methodology	13
9.1	Agile	13

1.0 Problem statement

The purpose of the requirements document is to systematically capture requirements for the project and the system “Policy Management System” to be developed. The application should be Cloud Native Architecture with Microservice. Both functional and non-functional requirements are captured in this document. It also serves as the input for the project scoping.

About the System

Policy Management System (PMS) is basically a system that helps to maintain the Policies and Payments. It also allows to search a Policy and allow end users to manage policies with ease.

The following section will cover aspects related to Bank Management System.

- a) User Registration.
- b) Policy Registration.
- c) Search Policies.

Scope of the System

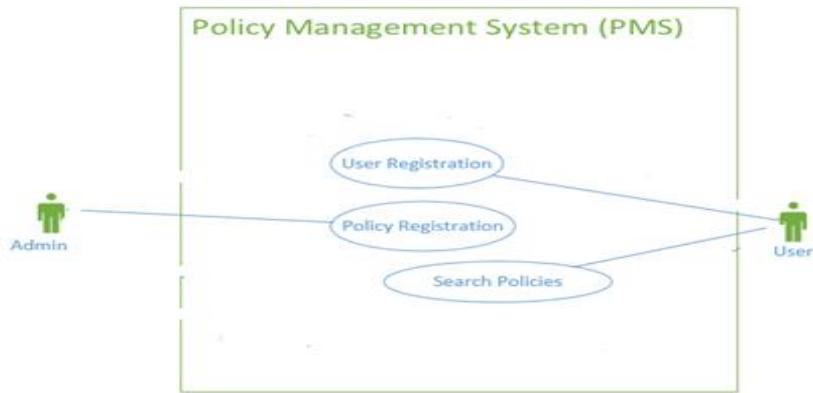
The scope of the system is explained through its modules as follows

The scope of the system is explained through its modules as follows

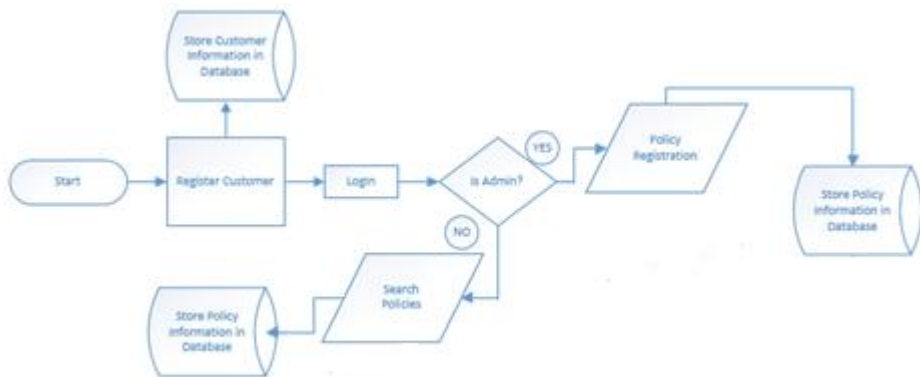
- User Registration - User should submit the details into the system.
- Policy Registration- Policy Registration be used by the admin to register the policies. The system will generate the policy id and if it is a valid policy the system will store the details in the database.
- Search Policies- Search policies. The system will provide a search engine mechanism to find the policies. This will access the database by getting the details based on user search criteria

2.0 Architecture Diagram

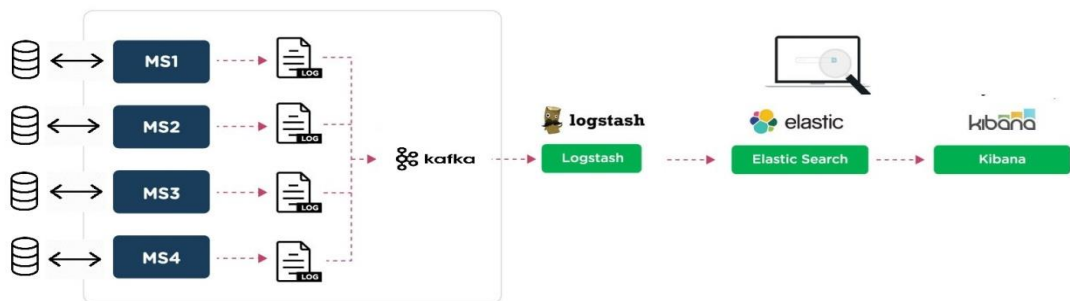
2.1 Use Case Diagram



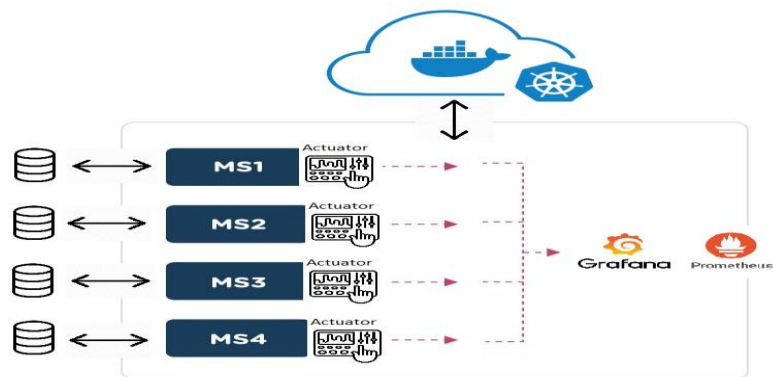
2.2 System Diagram



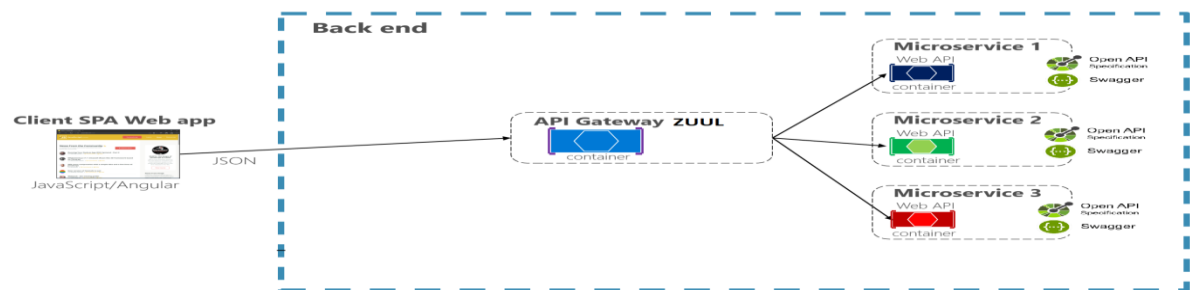
2.3 Microservice Architecture (Governance and Tooling/ Database and Storage):



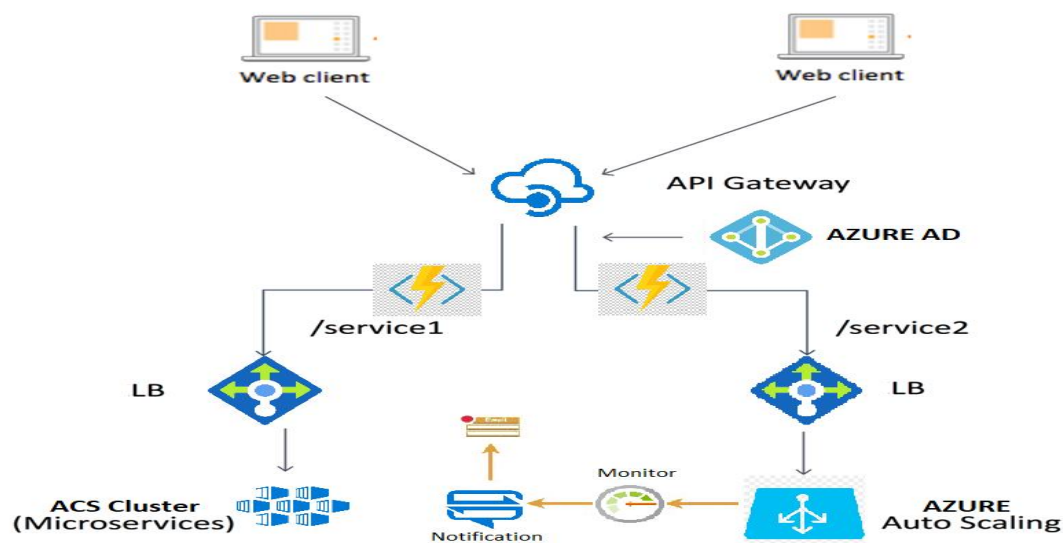
2.4 Microservice Architecture (Governance and Tooling/ Compute and Integration):



2.5 Microservice Architecture (Compute and Integration/Presentation/Networking and Content Delivery):



Cloud Architecture



3.0 Tool Chain

Associate will implement skills from Backend and azure Cloud platform to develop the application. Below are the skill details. Choose the Backend based on the technology stack.

Competency	Skill	Skill Detail
Engineering Mindset	Networking and Content Delivery	
	Ways of Working	
	Consulting Mindset	
	DevOps	
	Secure Coding	Veracode
	Code Quality	
Programming Languages	Application Language	Java
Products & Frameworks	Presentation	Angular
		Javascript/Typescript
	Networking and Content Delivery	Zuul
	Security and Identity	OpenIAM
	Compute & Integration	Spring Boot
		ELK Stack
		Docker
		Kubernetes
	Database & Storage	MongoDB
		MySQL
	Governance & Tooling	Git

		Junit
		Mockito
Engineering Quality		
Platform	Cloud Tools	AzureACS (Container Services)
		Azure CosmosDB/SQL DB
		Azure RedisCache/Storage
		Azure DevOps/Pipeline
		Azure API Gateway
		Azure Load Balancer
		Azure Notification Hubs
		Azure Functions

Development Flow

No	MC	Competency	Section
	Business Requirement		
1	Backend	Rest API, Database, Messaging System, Log/Monitoring, Identity and Access Management system, container orchestration, TDD, Debugging and Troubleshooting, automate repetitive workflows, Static and Dynamic Analysis	Click here
2	Front End	Angular/React, AJAX, Build & Deploy	Click here
3	Cloud	Compute, Identity, Compliance, Security and Content Delivery	Click here

4.0 User Stories

User Story #	User Story Name	User Story	Condition
--------------	-----------------	------------	-----------

US_01	Customer Registration	As a Customer, I should be able to register my details in the system Acceptance criteria: Customer should be able register the details in the system and it should be saved in the database. Capture the details below 1. First name - Name of the user 2. Last name -Name of the father 3. Date Of Birth- User DOB in DD/MM/YYYY format 4. Address- Address of the User 5. Contact no- Contact number of the Applicant 6. Email Address- Email Address of the User 7. Salary- Salary per month 8. PAN No- Pan Card Number 9. Employer type- Type of the Employer 10. Employer- Name of the Employer if Salaried or Optional For failed scenarios, System should display appropriate user friendly error messages. Note: <table><tr><th>Income per year (in lakhs)</th><th>User type</th></tr><tr><td>5</td><td>A</td></tr><tr><td>10</td><td>B</td></tr><tr><td>15</td><td>C</td></tr><tr><td>30</td><td>D</td></tr><tr><td>More than 30</td><td>E</td></tr></table> Business Rules and Validation: • Email id should not exist in the system. • If Employer type is self-employed, company should not be present. • If Employer type is salaried, company should be present.	Income per year (in lakhs)	User type	5	A	10	B	15	C	30	D	More than 30	E	Only API need to be developed
Income per year (in lakhs)	User type														
5	A														
10	B														
15	C														
30	D														
More than 30	E														

US_02	Policy Registration	As an admin user, I should be able to create a policy to the customers. Acceptance criteria: Admin users should be able to create a policy in the system and it should be saved in the database. Capture fields like 1. Policy name- Name of the Policy 2. Start Date- Start date in DD/MM/YYYY format 3. Duration in years- Duration in years 4. Company- Company name 5. Initial deposit- Amount to be deposited at start 6. Policy Type- Type of the policy 7. User Types- User types available for policy 8. Terms per year- Number of Payment Terms per year 9. Term amount- One term's payment amount 10. Interest- Interest for the policy System should generate the policy id and calculate the end date based on the duration provided. On successfully saving the details an email should be displayed in the format of html and the content as like below Dear Admin, The policy is successfully registered The policy <policy id> is available to the users from <policy start date> to <policy end date>. This is the <n>th policy in the <policy type>. To add more Click <Policy Registration>. Steps & Actions 1. System should get the all the required information from the admin.	Only API need to be developed

		2. Identify the eligibility based on the start date and duration provided. 3. Policy start date should be on or after current date 4. Policy types can be one of the below i. Vehicle Insurance ii. Travel Insurance iii. Life insurance iv. Health Insurance v. Child Plans vi. Retirement Plans 5. Generate the Policy id as follows. Policy id is <POLICYTYPEID>-<YEAROFPOLICYSTART>-XXX Fetch the last generated policy id to identify the new number. But the last generated policy id has to be retrieved separately from the database for each policy type. (Example: if the value for policy type is 'Child Plans' and start date is 15-07-2014 means that the last id generated in the policy type is CP-2014-005 in POLICIES table, then 005 will be taken and 1 will be added. So the next record's Policy id will be CP-2014-006.) 6. Generate the maturity amount as follows. $\text{Maturity amount} = (\text{initial deposit}) + (\text{duration} * \text{term_in_years} * \text{term amount}) + ((\text{duration} * \text{term_in_years} * \text{term amount}) * (\text{interest} / 100))$ Business Rules and Validation: Policy type needs to be selected. User types needs to be selected. End date and maturity amount should be calculated	
--	--	--	--

US_03	Search Policy	As a customer, I should be able to login and search the policies that have been created by admin Acceptance criteria: When customer logs in, he should be able to search policies based on the following criteria. 1. Number of years- Number of years 2. Company Name- Company Name 3. Policy Type- Type of the policy 4. Policy Id- Policy Id 5. Policy Name- Name of the Policy System should show the complete details about the policies. The search criteria should accept any one the following a. Policy Type b. Number of years c. Company Name d. Policy Id e. Policy Name Fetch the details of policies and show accordingly. If no policies found for the user display user friendly messages	Both API and Front end need to be developed
-------	---------------	---	---

5.0 Proposed Rest Endpoints to be exposed

5.1 Rest APIs:

POST	/api/v1.0/customer/register	Register a new user
POST	/api/v1.0/policy/register	Register a new policy
GET	/api/v1.0/policy /getall	Fetches all the policy Details

GET	/api/v1.0/policy/ searches	Fetches policy Detail based on Policy Type, Number of years, Company Name, Policy Id, Policy Name
[POST GET]	[/searches /searches]	[# create a new search # list all searches]

6.0 Rubrics/Expected Deliverables

6.1 Engineering Concepts (Compute & Integration):

1. As an application developer:
 - a. Develop the application as a microservice architecture.
 - b. Implementation as follows:
 - i. Use Domain Driven Design to implement distributed architecture
 - ii. Follow the Single Data Store per microservice practice
 - iii. Document REST endpoints with OpenAPI or Swagger
 - iv. Add CQRS pattern for Event Sourcing for saving and retrieving stock details
 - v. Expose all rest Endpoints using a common API Gateway Zuul

6.2 Engineering Concepts (Security & Identity):

1. As an Application Developer:
 - a. Restrict the access over all write operation (secured operations) by adding authentication
 - b. Secure all Rest EndPoints by configuring SSL Certificate for Cloud
 - c. Use OpenIAM to add OTP based two factor authentication for secured operations

6.3 Products & Framework (Database & Storage):

1. As an application developer:
 - a. Implement ORM with Spring Data MongoRepository and MongoDB. For complex and custom queries, create custom methods and use @Query, Aggregations (AggregationOperation, MatchOperation, AggregationResults), implementation of MongoTemplate etc as necessary.
 - b. Use MySQL for maintaining data for atleast one of the microservice
 - c. Introduce a backup mechanism, such that when record count crosses 10,000 rows, a backup should trigger

6.4 Products & Framework (Governance & tooling):

1. As an application developer
 - a. Containerize the complete application, using docker and Dockerfile.
 - b. Orchestrate container instances using kubernetes

- c. Use .dockerignore as necessary to avoid containerizing un-necessary packages.
- d. Integrate Spring Boot Actuator with Prometheus/Grafana to monitor middleware.
- e. Assemble logs from all microservice through kafka in Logstash.
- f. Pass on logstash data to ElasticSearch and organize them index. One index for each microservice.
- g. Pass this indexed data from ElasticSearch to Kibana and generate charts for each microservice logs

6.5 Debugging & Troubleshooting

1. Generate bug report & error logs - Report must be linked with final deliverables which should also suggest the resolution for the encountered bugs and errors.

6.6 Code Quality/Optimizations

1. Optimize MongoDB implementations using index search and then report the updated response time metrics.
2. Use Veracode to scan the backend application for security vulnerabilities, and submit the report of the same
3. Associates should have written clean code that is readable
4. Associate should have used the Code Analyzer (PMD/SonarQube) to ensure code quality and standard code style.

7.0 FrontEnd

- Develop the front end for user story
- Implement using either Angular or React
- Implement all the Front-End validation rules
- Proper naming conventions and folder structures
- Implement using proper SOLID design principles

8.0 Platform

8.1 Compute

1. Use ACS CLI (as an alternative to Azure Management Console) for container management and deployment of microservices. You should be able to explain and demonstrate the same in interview.
2. Use NoSQL instance of Azure CosmosDB/(SQL) as a database for the Market Application

8.2 Compute, Identity & Compliance, Security& Content Delivery

1. Use Azure Functions and Azure CosmosDB to build a backend process for handling requests for Market App.

2. Orchestrate containers with ACS to run the Microservices
3. Use Azure API Gateway to expose the built Azure functions to be accessible on public internet.
4. Use Azure LB to configure the load balancing of the instances.
5. Configure Azure Notifications to issue messages whenever Auto Scaling is launching or terminating the Container Instances in the Auto Scaling group.
6. Use Azure Security Centre for authentication.
7. Use Azure AD to Sign Up the user for secured operations.
8. Configure Azure Monitor to receive, maintain and expose metrics of all the write operations (Adding company and new stock price details)
9. Note : Minimum 3 API's to be hosted in cloud

9.0 Methodology

9.1 Agile

1. As an application developer, use project management tool along to update progress as you start implementing solution.
2. As an application developer, the scope of discussion with mentor is limited to:
 - a. Q/A
 - b. New Ideas, New feature implementations and estimation.
 - c. Any development related challenges
 - d. Skill Gaps
 - e. Any other pointers key to UI/UX and Middleware Development